

SOFTWARE ENGINEERING PRACTICE-I

MIDTERM REPORT

OpenNutri: LLM-Assisted Extraction of Food Data from Scientific Literature

Students:

- 221229078 - Arciel Aliognis Baez Zamora
- 221229075 - Duc Huan Ngo
- 221229031 - Ayşegül Doğan

Advisor: Dr. Öğr. Üyesi Fatma Zehra Solak

Exam Date: [To be filled by the advisor]

SUMMARY OF WORK COMPLETED DURING THE TERM

This midterm report summarizes the current state of the first three work items defined for the first semester in the project proposal:

- automated data acquisition pipeline,
- database and orchestration architecture,
- expert annotation engine.

By the midterm stage, the team has moved beyond a purely conceptual design and delivered a working system core. At this point OpenNutri already has an infrastructure that can discover candidate papers from scientific sources, filter them through multiple stages, transfer suitable PDF files into the system, let an expert user create structured annotations on those papers, and feed those annotation outcomes back into later crawling decisions.

The most important aspect of these outputs is that they do not exist as disconnected modules. They operate as a single flow centered on a shared data model. Papers found on the crawler side are transferred into the Supabase-based storage and database layer, the annotator interface presents these papers to the user, user actions are recorded as label events, and these labels are then converted by `feedback/update_terms.py` into new feedback terms that improve later crawler queries and ranking scores.

Table 1 summarizes how the current system state maps to the first-semester proposal items.

Proposal item	Midterm status	Concrete output
1. Automated Data Acquisition Pipeline	Substantially progressed	Multi-source search, language-scoped workflows, PDF acquisition, validation, upload to Supabase
2. Database and Orchestration Architecture	Substantially progressed	Shared schema, RLS policies, storage path, ETL and search-evidence tables
3. Expert Annotation Engine	Substantially progressed	PDF viewing, nutrient highlighting, dynamic food/nutrient entry, test mode, global skip, event logging
4. Document Segmentation and Core Extraction Process	Outside the scope of this report	A production-ready extraction pipeline is not claimed as completed yet

At the system level, the work completed during the term clusters around three main axes.

The first axis is data acquisition and candidate-paper stock creation. The crawler initially focused on Europe PMC, but it was later refactored into a source-agnostic Search -> Filter -> Acquisition architecture. This means a paper is now evaluated before any PDF download takes place, using signals such as title, abstract, language, source bias, food/nutrient matches, unit patterns, embedding similarity, and learned feedback terms. This approach reduces unnecessary PDF downloads and helps place more relevant candidates into the annotator queue.

The second axis is the shared backend and data model. When papers, annotations, `food_items`, `annotation_nutrient_values`, `paper_label_events`, `paper_global_labels`, `paper_search_hits`, `paper_search_batches`, and `paper_search_batch_hits` are considered together, the system stores not only final user data

but also how a paper was discovered and why it was accepted or rejected. This decision makes the system easier to evaluate technically and also creates the infrastructure needed for later experimental comparisons.

The third axis is the expert annotation interface. The React/Vite annotator is no longer just a simple form. It has become a real working environment that includes PDF rendering, nutrient highlighting inside the text layer, quick nutrient insertion from highlights, ranked food and nutrient lookup, test mode, a global "definitely no data" flow, and user-action event logging. The changes in `Annotate.jsx`, `PdfViewer.jsx`, and `PdfTextScanner.js` show that the user-facing side of the project has moved from research prototype level toward a usable tool.

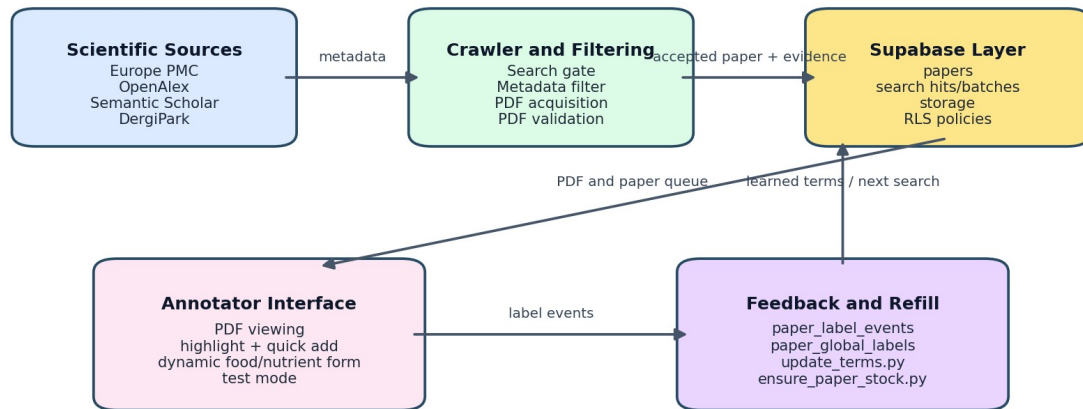
The term also included quality and consistency fixes, not only new features. For example, only valid food items are now counted and saved so that empty placeholder cards do not create incorrect totals. Likewise, the `food_item_count` and `nutrient_value_count` fields stored in `paper_label_events` now reflect actual user output more accurately. On the crawler side, dropping the legacy `seen_ids` logic in favor of `paper_states` also improves reproducibility and control over repeated runs.

Table 2 lists the high-impact developments that were added to the repository after the handoff snapshot.

Date	High-impact development	Project impact
2026-03-20	Cumulative and field-aware soft feedback learning	Terms learned from labels started feeding back into crawler scoring
2026-03-21	Split English and Turkish workflows	EN/TR sources began to be managed with separate language-specific targets
2026-03-22	`Search -> Filter -> Acquisition` refactor and search-evidence tables	Metadata-level filtering and query-evidence storage became possible before PDF download
2026-03-30	Annotator count fix and canonical hit deduplication	UI data quality and crawler evidence consistency improved
2026-03-30	Turkish crawl quotas, metadata-only hit persistence, DergiPark index refresh	More controlled and traceable Turkish-literature acquisition was established
2026-03-30	Query-batch feedback and search-gate batch accounting	It became measurable which query batches yield better results
2026-03-30	Removal of hard-negative veto logic	Crawler decisions became more balanced through soft penalties instead of hard vetoes

Figure 1 shows the end-to-end system state available at the midterm stage.

OpenNutri Midterm Architecture



Closed-loop logic: expert labels are not only a final output; they also influence later search and ranking decisions.

Figure 1. Closed-loop OpenNutri architecture implemented by the midterm stage.

The key point to emphasize here is that the project has not yet completed every item in the proposal, but the first three goals are no longer only planned. They already exist as working system components. For that reason, the current stage should be evaluated not as a vague middle ground between "design document" and "finished product," but as an early production-stage system whose core infrastructure is already operating in practice.

PROJECT OBJECTIVE and IMPORTANCE

Project Objective

The objective of OpenNutri is to develop a platform that digitizes food-composition data scattered across scientific literature without removing human expertise from the loop; instead, expert feedback is treated as a central component of the system. To achieve that goal, the project aims to establish three capabilities at the same time:

- automatically finding relevant papers and bringing them into the system,
- allowing experts to annotate those papers in a controlled way,
- using the resulting labels to improve later retrieval and ranking decisions.

At the midterm stage, the focus is to build the core version of these three capabilities in line with the first-semester goals. Accordingly, the current work does not claim that a full LLM-based extraction system has already been completed. Instead, it focuses on building the data and user infrastructure that such a system could later rely on for training and validation.

Project Importance

The importance of the project appears at three levels.

The first level is the data-access problem. Food-composition studies are usually published as PDF documents, distributed across different journal infrastructures, and often lack any standard machine-readable output format. In other words, the information exists, but it cannot be used directly as queryable structured data. OpenNutri targets this gap.

The second level is efficient use of expert time. Expert annotation is expensive and limited. If irrelevant papers reach the annotator queue, expert effort is wasted. For this reason, the project is not only about building a UI; it also tries to improve what reaches the expert by combining the crawler and feedback layers. At the midterm stage, the implemented search gate, metadata filter, PDF validation, and global skip logic directly support this goal.

The third level is the visibility of Turkish literature. The proposal explicitly targets PubMed and DergiPark. This choice is deliberate. PubMed Central provides access to international open-access literature, while DergiPark gives access to many studies published in Turkey. The shift toward language-scoped EN/TR workflows is therefore important because Turkish studies are no longer treated as a side effect; they are treated as a distinct target pool.

From a technical point of view, the project is important not because it uses a single advanced algorithm, but because it combines a normalized data model, multi-source search, layered filtering, PDF validation, user-event logging, and soft feedback learning in one system. This integrated approach goes beyond a conventional CRUD application and creates a strong base for later document segmentation and LLM-based extraction work.

LITERATURE REVIEW

The project was designed by examining both food-data standards and scientific-document processing tools together. The literature review therefore did not rely only on academic papers; it also considered databases, open-access archives, ontologies, and the software ecosystems used in the implementation.

1. Food data and reference vocabularies

OpenNutri's data model was not built directly on free-text labels. FoodData Central [1] and the FAO/INFOODS matching approach [2] were reviewed, and the concepts of canonical foods and canonical nutrients were modeled as separate tables. As a result, the food and nutrient choices visible in the UI are tied to a shared vocabulary that can later be reused both for crawler term generation and for validating any future LLM extraction outputs.

Ontology-centered work such as FoodOn [3] is also relevant, especially for standardizing food names. The current project version does not implement a full ontology integration, but the decision to use entities and `entity_aliases` was motivated by the same need for controlled standardization.

2. Scientific literature sources

PubMed Central [4] and Europe PMC [5] were evaluated as core external sources because they provide structured access to open-access biomedical and life-science literature. DergiPark [6] was included specifically to reach Turkish-language studies. During the midterm period, the DergiPark path was redesigned from a simple broad search approach into a renewable local journal/issue/article index. That change made Turkish-language crawling more controlled and auditable.

3. Human-in-the-loop learning approach

The project intentionally avoids fully automating every decision. Human-in-the-loop methodology [7] argues that experts should remain an active part of scientific extraction and verification workflows. In OpenNutri, this idea becomes concrete through the annotator UI and `paper_label_events`. User actions such as `draft`, `done`, `skipped`, and `global_definitely_no_data` are not just interface interactions; they also become feedback signals that influence later crawling.

4. PDF processing and UI infrastructure

Because article content is usually distributed as PDF, in-browser PDF processing became essential. PDF.js [8] and React [9] were therefore evaluated together. However, the PDF problem is not limited to rendering. Since the PDF text layer is fragmented across spans, the nutrient-highlighting feature required additional scanning, overlap resolution, and click-recovery logic. This highlights an important difference between literature-annotation tools and ordinary web forms.

5. Platform components used in the implementation

Supabase [10] was evaluated not merely as a database, but as a single platform layer providing authentication, row-level access control, file storage, and client access. At the midterm stage both the annotator and the data pipeline already share this backend. This choice was meaningful both for data consistency and for fast prototyping.

The literature review led to several engineering decisions:

- not combining paper discovery and PDF download into a single step,
- designing the UI with a dynamic row model instead of fixed nutrient columns,
- storing labels as event history rather than only a final state,
- handling Turkish and English sources under the same system but with separate targets,
- using feedback as a soft score instead of a hard rejection rule.

All of these decisions are already reflected in the current codebase, which means the project has moved from a theoretical design into an implemented system behavior.

MATERIALS AND METHODS

1. Overall system approach

The midterm version of OpenNutri consists of three major layers:

- the user-facing annotator interface,
- the shared backend and data model,
- the multi-source crawler and feedback layer.

The interaction of these layers was shown in Figure 1. Figure 2 expands that view by focusing on the internal data-model and feedback relationships.

Annotation and Feedback Data Relations

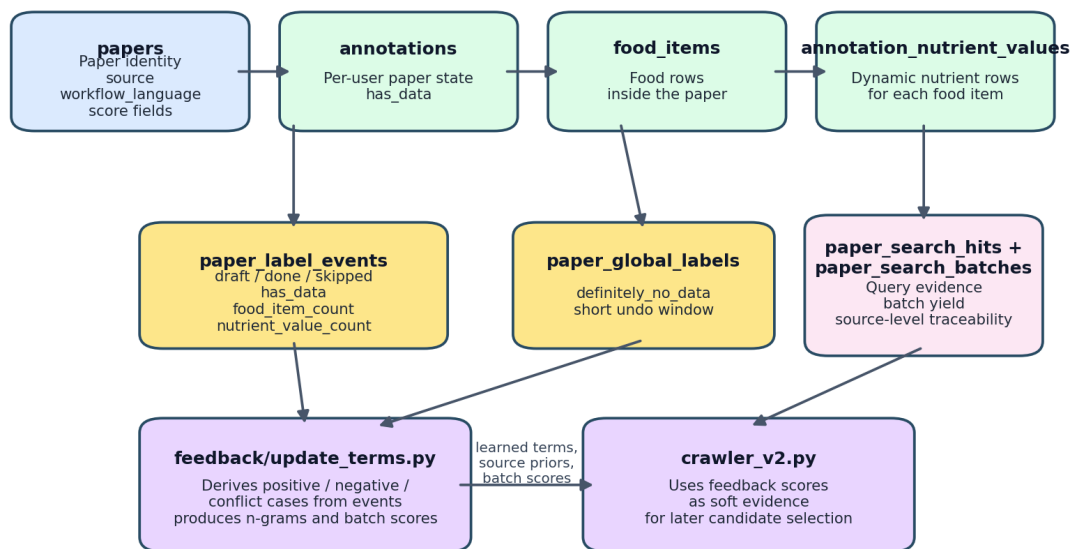


Figure 2. Data-model relationships linking annotation, event logging, and crawler feedback in the midterm system.

2. Materials used

The main technical materials used in the project are:

- **Frontend framework:** React 19 + Vite
- **Backend and storage:** Supabase Auth, PostgreSQL, Storage
- **PDF handling:** react-pdf and PDF.js
- **Data-pipeline language:** Python
- **Reference data source:** USDA FoodData Central [1]
- **Paper sources:** Europe PMC [5], PubMed Central [4], OpenAlex, Semantic Scholar, DergiPark [6]
- **Embedding layer:** a dual English + multilingual setup based on sentence-transformers

These components are used across two connected sides of the project: the web application that manages expert workflow and the data pipeline that feeds the paper queue.

3. Backend and data-model method

On the backend side, the team first established a normalized reference-data schema. The `entities`, `entity_aliases`, `master_nutrients`, `sources`, and `claims` tables represent the food and nutrient vocabulary. This layer is intentionally separated from user annotations, which allows both the frontend autocomplete components and crawler term-generation logic to reuse the same controlled vocabulary.

On top of that, a second relational layer was built for annotation. The `papers` table stores the imported paper records, `annotations` stores per-user paper state, and `food_items` plus `annotation_nutrient_values` store flexible food/nutrient entries. A dynamic row-based structure was preferred over a rigid nutrient panel. As a result, one paper may contain only proximate composition while another may include vitamins or minerals without the model becoming artificially restrictive.

During the midterm period the backend was expanded in two important ways. First, `paper_label_events` and `paper_global_labels` were added so user actions could be stored as learning-oriented event data. Second, `paper_search_hits`, `paper_search_batches`, and `paper_search_batch_hits` were added so the crawler stores not only accepted papers but also the search evidence that led to them. This makes it possible to answer questions such as "which query combinations worked better?" directly from the database.

Row-level security (RLS) is used to protect the backend. Users can manage their own annotation data, while the service role remains able to perform crawler uploads, ETL, and maintenance operations. This is the core security mechanism required by the system's multi-user design.

4. Annotator interface method

The center of the annotator interface is `Annotate.jsx`. When the application loads, this component fetches papers, the current user's saved annotation states, the nutrient reference list, and the food catalog. When the selected paper changes, any previous annotation for that paper is restored so the user can continue from the last saved point.

The UI uses a dynamic form model for food and nutrient entry. Each food item can contain an arbitrary number of nutrient rows. `FoodAutocomplete.jsx` and `NutrientAutocomplete.jsx` rank candidates using exact matches, prefix matches, token normalization, and alias handling. This means the user does not need to type the exact canonical database name every time.

For PDF interaction, `PdfViewer.jsx` and `PdfTextScanner.js` work together. The text-layer spans produced by `PDF.js` are scanned, nutrient terms are wrapped with `<mark>` where appropriate, and the user can click those highlights to open a quick nutrient-entry popover. This turns the interface from a plain data-entry form into a document-aware annotation tool.

Three interface behaviors were particularly added or improved during the midterm period:

- a test mode that allows safe use of the full UI without writing to the real database,
- a global "definitely no data" action with a short undo window,
- counting only valid food items so that empty placeholder cards do not create incorrect `food_item_count` values.

Figure 3 was generated to show that the crawler now produces staged and measurable funnel outputs.

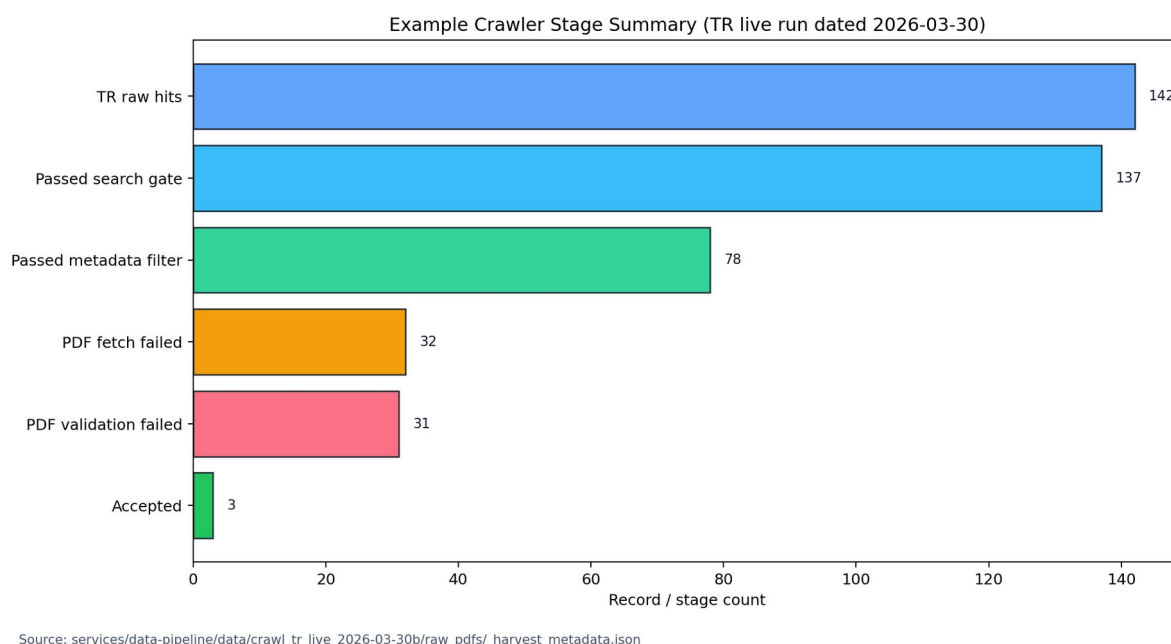


Figure 3. Stage counts derived from the manifest summary of the example Turkish live run dated 2026-03-30. This figure is not presented as a full benchmark; it is included to show that the pipeline already produces measurable staged behavior.

For the real annotator UI screenshot, Figure 4 is intentionally left as a placeholder.

Placeholder for Figure 4

This visual should be replaced with a real annotator screenshot

1. In apps/expert-annotator run `npm install && npm run dev`.
2. Sign in and open the Annotate page.
3. Make sure the same frame includes:
PDF viewer, a highlighted nutrient, the right-side food item form,
and the progress / status area near the top.
4. Save the screenshot under the same filename as
`docs/defense/assets/figure_4_annotator_placeholder_en.png`
and rerun the export script.

Figure 4. Before final submission, this visual should be replaced with a real screenshot of the current annotator interface. The simplest approach is to replace `docs/defense/assets/figure_4_annotator_placeholder_en.png` with the screenshot under the same filename and rerun the export script. The image should include the PDF viewer, an example highlighted nutrient, the food-item form, and the progress/status area in the same frame.

5. Crawler, filtering, and acquisition method

The crawler was not designed as a one-step search script. By the midterm stage, the implemented approach has three stages:

- **Search:** finding metadata-level candidates from sources such as Europe PMC, OpenAlex, Semantic Scholar, and DergiPark
- **Filter:** applying search-gate and metadata-filter logic to titles and abstracts
- **Acquisition:** downloading PDFs and validating full text only for sufficiently strong candidates

This separation is an important engineering decision. Downloading every candidate PDF would be both expensive and unnecessary. The pre-filtering step allows fewer but stronger candidates to move to full-text acquisition.

The filtering logic combines several signals:

- composition- and nutrient-content-oriented lexical patterns,
- unit signals such as mg/100g and g/100g,
- food-term and nutrient-term hits,
- negative signals related to health outcomes and clinical contexts,
- English and multilingual embedding similarity,
- feedback terms derived from user labels,
- source-level priors and query-batch yield information.

Near the end of the midterm period, the crawler was expanded in two major directions. First, English and Turkish workflows were split so that acceptance targets are managed per language. Second, a query - batch feedback mechanism was added, allowing the system to score not only which terms are useful but also which query batches tend to lead to better labeled outcomes.

For Turkish-language acquisition, the DergiPark integration was also redesigned. Instead of a broad and weakly controlled search approach, the crawler now uses renewable journal- and issue-level local index files. This improves both source quality and traceability for Turkish literature.

6. Feedback and paper-stock refill method

The feedback/update_terms.py script reads the paper_label_events and paper_global_labels records created by users and derives training labels. The current logic is:

- draft or done counts as positive only when has_data=true, food_item_count>0, and nutrient_value_count>0,
- a global definitely_no_data label or skip signals from at least two different users count as negative,
- conflicting cases are excluded from training.

From these examples, the system extracts n-gram terms at both title-only and title+abstract levels, compares positive and negative distributions, and produces query phrases, anchor phrases, weighted terms, source priors, and batch scores. The crawler then consumes those outputs as soft scores rather than hard veto rules. In other words, feedback is treated as a balanced scoring signal instead of an irreversible rejection.

When the end-user paper pool becomes too small, ensure_paper_stock.py takes over. This script checks the current EN/TR paper counts, refreshes feedback when needed, refreshes the DergiPark index, runs the crawler, and uploads the results to Supabase. That creates an operational bridge between the annotation interface and the data-acquisition layer.

7. Current limitations

At the midterm stage, proposal item four, document segmentation and an LLM-based core extraction process, is not presented as a completed production pipeline. The repository contains exploratory and prototype-level files in that direction, but this report is intentionally limited to the first three working semester goals and the data/feedback infrastructure supporting them.

Similarly, the real annotator screenshot is still left as a placeholder in this report. The reason is practical: it is better to capture the most up-to-date UI state close to final submission.

REFERENCES

1. U.S. Department of Agriculture. FoodData Central. <https://fdc.nal.usda.gov/>
2. FAO/INFOODS. Guidelines for Food Matching. Rome: Food and Agriculture Organization of the United Nations, 2012.
3. Dooley DM, Griffiths EJ, Gosal GS, et al. FoodOn: a harmonized food ontology to increase global food traceability, quality control and data integration. **npj Science of Food**. 2018;2(1):23.
4. National Center for Biotechnology Information. PubMed Central (PMC). <https://pmc.ncbi.nlm.nih.gov/>
5. Europe PMC. <https://europepmc.org/>
6. DergiPark Akademik. <https://dergipark.org.tr/>
7. Monarch RM. **Human-in-the-Loop Machine Learning**. Manning Publications; 2021.
8. Mozilla. PDF.js. <https://mozilla.github.io/pdf.js/>
9. React Documentation. <https://react.dev/>
10. Supabase Documentation. <https://supabase.com/docs>